EXERCISE 5.2

Write the function `take(n)` for returning the first `n` elements of a `Stream`, and `drop(n)` for skipping the first `n` elements of a `Stream`.

EXERCISE 5.3

Write the function `takeWhile` for returning all starting elements of a `Stream` that match the given predicate.

```
def takeWhile(p: A => Boolean): Stream[A]
```

You can use `take` and `toList` together to inspect streams in the REPL. For example, try printing `Stream(1,2,3).take(2).toList`.

5.3 Separating program description from evaluation

A major theme in functional programming is *separation of concerns*. We want to separate the description of computations from actually running them. We've already touched on this theme in previous chapters in different ways. For example, first-class functions capture some computation in their bodies but only execute it once they receive their arguments. And we used `Option` to capture the fact that an error occurred, where the decision of what to do about it became a separate concern. With `Stream`, we're able to build up a computation that produces a sequence of elements without running the steps of that computation until we actually need those elements.

More generally speaking, laziness lets us separate the description of an expression from the evaluation of that expression. This gives us a powerful ability—we may choose to describe a "larger" expression than we need, and then evaluate only a portion of it. As an example, let's look at the function `exists` that checks whether an element matching a `Boolean` function exists in this `Stream`:

```
def exists(p: A => Boolean): Boolean = this match {
  case Cons(h, t) => p(h()) || t().exists(p)
  case _ => false
}
```

Note that `||` is non-strict in its second argument. If `p(h())` returns `true`, then `exists` terminates the traversal early and returns `true` as well. Remember also that the tail of the stream is a `lazy val`. So not only does the traversal terminate early, the tail of the stream is never evaluated at all! So whatever code would have generated the tail is never actually executed.

The `exists` function here is implemented using explicit recursion. But remember that with `List` in chapter 3, we could implement a general recursion in the form of `foldRight`. We can do the same thing for `Stream`, but lazily:

```
def foldRight[B](z: => B)(f: (A, => B) => B): B =
  this match {
    case Cons(h,t) => f(h(), t().foldRight(z)(f))
    case _ => z
  }
```

If `f` doesn't evaluate its second argument, the recursion never occurs.

The arrow `=>` in front of the argument type `B` means that the function `f` takes its second argument by name and may choose not to evaluate it.

This looks very similar to the `foldRight` we wrote for `List`, but note how our combining function `f` is non-strict in its second parameter. If `f` chooses not to evaluate its second parameter, this terminates the traversal early. We can see this by using `foldRight` to implement `exists`:[5]

```
def exists(p: A => Boolean): Boolean =
  foldRight(false)((a, b) => p(a) || b)
```

Here `b` is the unevaluated recursive step that folds the tail of the stream. If `p(a)` returns `true`, `b` will never be evaluated and the computation terminates early.

Since `foldRight` can terminate the traversal early, we can reuse it to implement `exists`. We can't do that with a strict version of `foldRight`. We'd have to write a specialized recursive `exists` function to handle early termination. Laziness makes our code more reusable.

EXERCISE 5.4

Implement `forAll`, which checks that all elements in the `Stream` match a given predicate. Your implementation should terminate the traversal as soon as it encounters a nonmatching value.

```
def forAll(p: A => Boolean): Boolean
```

EXERCISE 5.5

Use `foldRight` to implement `takeWhile`.

EXERCISE 5.6

Hard: Implement `headOption` using `foldRight`.

[5] This definition of `exists`, though illustrative, isn't stack-safe if the stream is large and all elements test `false`.

EXERCISE 5.7

Implement `map`, `filter`, `append`, and `flatMap` using `foldRight`. The `append` method should be non-strict in its argument.

Note that these implementations are *incremental*—they don't fully generate their answers. It's not until some other computation looks at the elements of the resulting `Stream` that the computation to generate that `Stream` actually takes place—and then it will do just enough work to generate the requested elements. Because of this incremental nature, we can call these functions one after another without fully instantiating the intermediate results.

Let's look at a simplified program trace for (a fragment of) the motivating example we started this chapter with, `Stream(1,2,3,4).map(_ + 10).filter(_ % 2 == 0)`. We'll convert this expression to a `List` to force evaluation. Take a minute to work through this trace to understand what's happening. It's a bit more challenging than the trace we looked at earlier in this chapter. Remember, a trace like this is just the same expression over and over again, evaluated by one more step each time.

Listing 5.3 Program trace for `Stream`

```
Stream(1,2,3,4).map(_ + 10).filter(_ % 2 == 0).toList
cons(11, Stream(2,3,4).map(_ + 10)).filter(_ % 2 == 0).toList
Stream(2,3,4).map(_ + 10).filter(_ % 2 == 0).toList
cons(12, Stream(3,4).map(_ + 10)).filter(_ % 2 == 0).toList
12 :: Stream(3,4).map(_ + 10).filter(_ % 2 == 0).toList
12 :: cons(13, Stream(4).map(_ + 10)).filter(_ % 2 == 0).toList
12 :: Stream(4).map(_ + 10).filter(_ % 2 == 0).toList
12 :: cons(14, Stream().map(_ + 10)).filter(_ % 2 == 0).toList
12 :: 14 :: Stream().map(_ + 10).filter(_ % 2 == 0).toList
12 :: 14 :: List()
```

Apply `filter` to the first element.

Apply `map` to the first element.

Apply `map` to the second element.

Apply `filter` to the second element. Produce the first element of the result.

Apply `filter` to the fourth element and produce the final element of the result.

`map` and `filter` have no more work to do, and the empty stream becomes the empty list.

The thing to notice in this trace is how the `filter` and `map` transformations are interleaved—the computation alternates between generating a single element of the output of `map`, and testing with `filter` to see if that element is divisible by `2` (adding it to